Mid-Year Report
Project #43: Reconfigurable Neural Processing Unit (NPU) for Energy-Efficient AI at the Edge

Supervisor: Morteza Biglari-Abhari
Co-Supervisor:Maryam Hemmat
Project Partner: Pratham Chhabra

Friday 25th July 2025

1 RESEARCH PLAN

Further literature reviews have shifted the project's focus on exploring methods for mitigating sparse matrices and improving on hardware utilisation. From the systolic array research, its structured data flow reduces the amount of memory access, however, applying MAC operations on a sparse matrix worsens the hardware utilisation. The aim for a reconfigurable NPU remains relevant by having adaptable hardware architecture for improved energy-efficiency.

2 CURRENT IMPLEMENTATION

2.1 System Overview

The system plan is shown in Figure 1 where the current implementation consists of a control unit and a systolic array (acting like the data path) to simulate a basic NPU. The trained machine-learning model provides provides the inputs, as data and weights NxN matrices, to the NPU for inference. The control unit analyses the input and decides how the matrices should be fed into the systolic array through a staggering input method and then the systolic array performs MAC operations. Currently, the output of the systolic array does not get saved into an allocated memory as all work is done through simulation on ModelSim.

2.2 Systolic Array

Systolic arrays are utilised in this project because of its high reuse data rate. Instead of repeatedly fetching data from memory, data is passed between neighbouring processing elements (PEs) through dedicated interconnections. This significantly reduces data movement, which in turn lowers energy consumption and latency. This means that memory is only accessed at the boundaries of the array.

Currently, the data flows through an orthogonal 2D NxN systolic array of PEs. Each PE receives a data and weight value and performs a multiply-accumulate (MAC) operation. This can be demonstrated in Figure 2. After computing the MAC operation, it passes the data and weight values to the corresponding neighbouring PE, as shown in Figure 3.

Each MAC operation takes only one clock cycle because the computation is independent from the data movement where the PE performs the operation in the same cycle that it receives the input. So

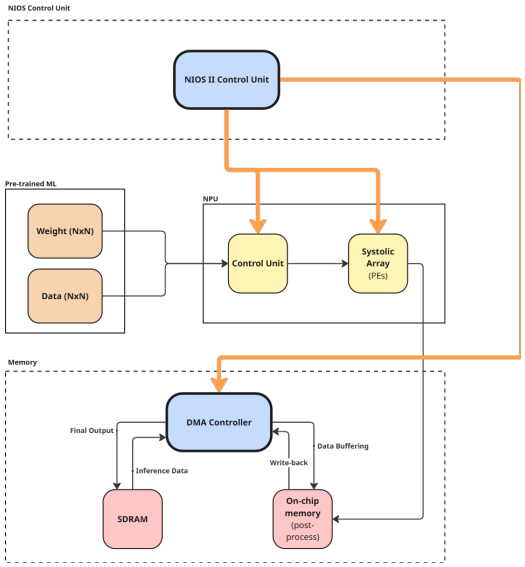

Figure 1: Full System Overview Plan

far, the systolic array has been simulated for sizes up to 8x8, where $0 < N \leq 8$. PEs are selectively enabled or disabled to match the input matrix size.

Although research has proposed improved data flows that reduce the total number of clock cycles, these have not yet been implemented in the design.

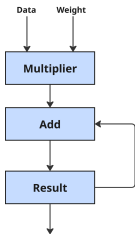

Figure 2: Hardware Architecture of Multiply-Accumulate (MAC) Operation

2.3 Control Unit

The control unit is responsible for enabling the appropriate PEs and controlling how input data is injected into the systolic array. To ensure that each PE receive the correct operands at the correct time to perform the MAC operation properly, the inputs must be staggered as it enters the array. As a result of the dataflow pattern, the total number of clock cycles required to complete a NxN matrix multiplciation is 3N-2. This value was initially calculated manually based on the observeed data movement and PE activiation timing, and later confirmed by a reference implementation in a paper. This timing behaviour is illustrated in Figure 4 where it shows how inputs enter the systolic array.

2.4 Verification

The system was tested using ModelSim simulation with two unique input matrices with on sparsity to validate its functionality. The testbench was configured to run at 50MHz, corresponding to the standard clock frequency of the DE1-SoC board. This was chosen as a reference due to familiarity however it has not been confirmed if this will be used in the final implementation. During simulation,

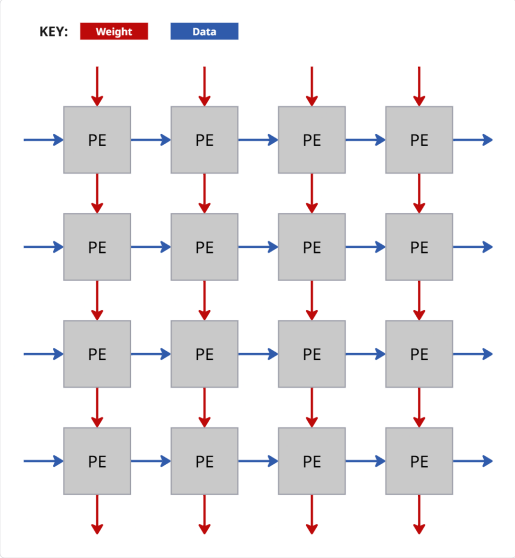

Figure 3: Dataflow of an 4x4 Systolic Array composed of Processing Elements

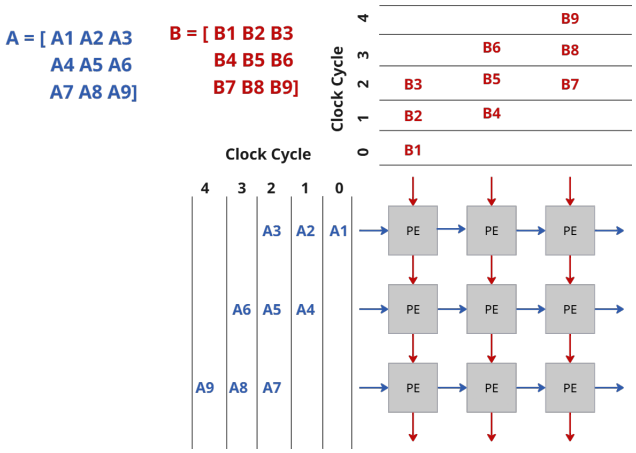

Figure 4: Staggered input to the Systolic Array

the output from ModelSim matched exactly the results expected generated by MATLAB, confirming correctness (shown in Figure 5 and 6). Additionally, the waveform analysis verified that each MAC operation completes in a single clock cycle as designed.

```
Result C = A * B:
        1704        1740        1776        1812        1848        1884        1920        1956
        4136        4236        4336        4436        4536        4636        4736        4836
        6568        6732        6896        7060        7224        7388        7552        7716
        9000        9228        9456        9684        9912       10140       10368       10596
       11432       11724       12016       12308       12600       12892       13184       13476
       13864       14220       14576       14932       15288       15644       16000       16356
       16296       16716       17136       17556       17976       18396       18816       19236
       18728       19212       19696       20180       20664       21148       21632       22116
```

Figure 5: Result of A*B in MATLAB used as a reference for correctness.

A timing analysis in Quartus showed a maximum achievable clock frequency of 118.64MHz (shown in Figure 7), exceeding 100MHz ensuring that the system can safely operate at 50MHz. However, when compiling the design to the DE1-SoC, a limitation was encountered due to the insufficient number of I/O pins, which restricts the board to deploying a 2x2 systolic array. Despite this constraint, once the compilation was adjusted, the system met the timing requirement and the ModelSim simulation results remained valid.

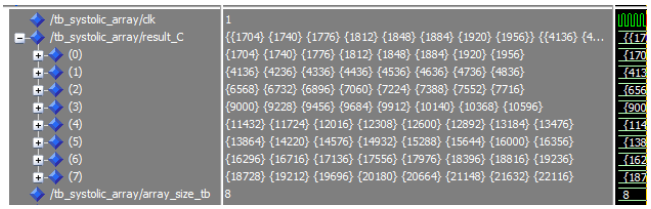

Figure 6: Output of ModelSim matching the result of the MATLAB code.

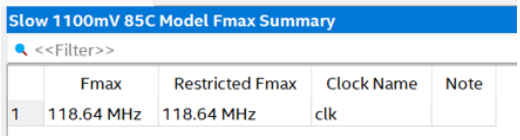

Figure 7: Timing Analysis of 2x2 Systolic Array running at 50MHz on DE1-SoC.

3 FUTURE IMPLEMENTATION

To improve the system's energy efficiency and to address the constraints found, future implementations are planned ahead.

3.1 Optimisation for Sparse Matrices

The literature revealed that one of the causes of sparsity in Convolutional Neural Networks (CNN) is the vanishing gradient problem. This issue can be mitigated by using activation functions such as the Rectified Linear Unit (ReLU). ReLU introduces non-linearity to avoid the gradient becoming zero but introduces sparsity as it modifies negative values to zeroes. To reduce sparsity, pruning techniques can be applied. From the literature, it has been proven that pruning the least significant weights and compacting them into smaller, denser, equally-sized matrices can significantly reduce computational overhead whilst preserving inference accuracy.

3.2 Scalability and Power Management

The concept of tiling will be implemented to support large input matrices like images. By partitioning the large matrices into smaller blocks that is processed simultaneously, energy efficiency and parallelism are improved, memory bandwidth is reduced, and allows for scalability. For power optimisation, power gating will be introduced in the hardware. Selective PE enabling has already been implemented in simulation but needs to be translated into hardware. This helps conserve energy dynamically by switching off idle PEs.

3.3 Memory System and Control

Currently, data movement is managed entirely within simulation without a physical memory implementaiton. To address this and mitigate CPU overhead, a NIOS II processor is integrated as a co-processor together with a DMA controller. NIOS will be responsible for:

- Controlling input/output data movement.
- Interfacing with the hardware accelerator and loading matrices into shared memory.
- Programming DMA for efficient data transfer between memory and the accelerator.

- Setting and managing the system's reconfiguration parameters.

4 CONCLUSION

In conclusion, substantial progress has been achieved in the analysis of relevant literature, enabling a successful implementation of a foundational systolic array and control unit. Verification has confirmed that Multiply-Accumulate (MAC) operations complete within a single clock cycle, important for performance metric. Subsequent steps will involve the integration of NIOS II processor as a co-processor, the implementation of a memory system into hardware, and the application of sparsity mitigation technique to optimise system efficiency.